

1) Sistemas Operativos - Generalidades

- ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?
- Explique a que se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?
- ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real? (Pag.95)

2) Kernel de FreeRTOS

- Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.
- Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.
- ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a que nos referimos con el concepto de “tareas manejadas por eventos”?
- ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.
- ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM? (Pag. 91)

6) Explicar el funcionamiento del esquema de la fig 1:

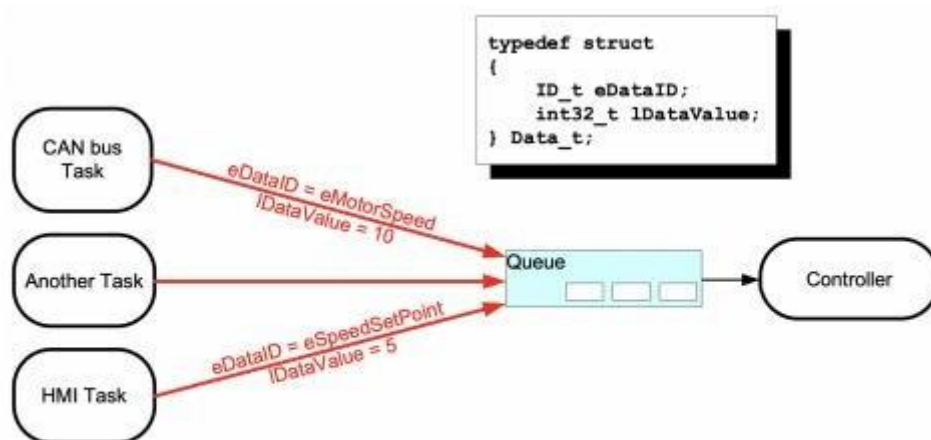


Fig. 1. Recibir datos de múltiples fuentes

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

- 8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.
- 9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.
- 10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Respuestas:

- 1)
- a) El encargado de administrar el tiempo de la CPU es el planificador (scheduler). Su función principal es decidir, en cada instante, qué tarea se ejecuta y durante cuánto tiempo, aplicando una política de planificación (Round-Robin en el caso de FreeRTOS). El objetivo es maximizar el uso del procesador y garantizar que las tareas críticas reciban el tiempo de CPU necesario, manteniendo el sistema en un estado eficiente y predecible.
 - b) El contexto de ejecución es el conjunto completo de información que define el estado de una tarea en un momento dado. Cuando el planificador cambia de una tarea a otra, debe guardar el contexto de la tarea actual y restaurar el de la siguiente para reanudar su ejecución sin errores. Las variables que intervienen son:
 - Registros internos del microprocesador (propósito general, puntero de pila, etc.).
 - Contador de programa (PC).
 - Pila de llamadas (stack), que contiene variables locales, direcciones de retorno y marcos de función.
 - c) Un Sistema Operativo de Tiempo Real (RTOS) está diseñado para ejecutar tareas con restricciones temporales estrictas; su comportamiento debe ser determinista y predecible. Se utiliza cuando el incumplimiento de un plazo puede causar fallos en el sistema (por ejemplo, control de motores, sistemas médicos, automoción). Los RTOS suelen ser ligeros, prescinden de protecciones innecesarias para ahorrar recursos y permiten la planificación por prioridades, lo que los hace ideales para sistemas embebidos con microcontroladores.

- 2)
- a) Las tareas se implementan como funciones de C. Solo se distinguen por su prototipo, que debe retornar vacío (void) y tomar un parámetro de puntero vacío (void *).

Ejemplo de prototipo: void Task_algo(void *pvParameters);

Cada tarea es un pequeño programa en sí mismo y siempre debe cumplir con las siguientes características

- La tarea nunca debe retornar (no usar return ni llegar al final). Debe contener un bucle infinito o eliminarse explícitamente con vTaskDelete(NULL).

- Cada tarea es una instancia independiente con su propia pila y copia de variables automáticas.
- Se crean múltiples tareas a partir de la misma función usando `xTaskCreate()`, pasando diferentes parámetros.

Ejemplo de tarea:

```

1  // Prototipo de la tarea
2  void vTaskEjemplo(void *pvParameters);
3
4  void vTaskEjemplo(void *pvParameters) {
5      // Configuración e inicialización (se ejecuta una sola vez)
6
7      while(1) { // Bucle infinito obligatorio
8          /* Lee un sensor de temperatura */
9          /* Envía el valor por una cola a otra tarea */
10
11          // Bloquea hasta por 1s para luego volver a leer
12      }
13  }
14
15  // Creación de la tarea (en main o en otra tarea)
16  xTaskCreate(
17      vTaskEjemplo,      // Función
18      "Tarea1",          // Nombre legible
19      1024,              // Tamaño de pila
20      NULL,              // Parámetro pasado a la tarea
21      1,                 // Prioridad
22      NULL               // Handler (no se usa)
23  );

```

b) Cada tarea puede estar en uno de los siguientes tres estados:

Ejecución (Running): El microprocesador la está ejecutando. Sólo puede haber una tarea en este estado al mismo tiempo por core físico (FreeRTOS usado por la cátedra en espresif IDF).

Bloqueado (Blocked): es un subestado de “Not running”, se dice que una tarea está bloqueada si está a la espera de un evento.

Suspendido (Suspended): El estado suspendido es también un subestado de “Not running”. Las tareas que se encuentran en estado suspendido no están disponibles para el Scheduler. La única forma de entrar en el estado suspendido es mediante una llamada a la función de API `vTaskSuspend()`. La única manera de salir del estado suspendido es mediante otra llamada a las funciones de API `vTaskResume()` o `xTaskResumeFromISR()`. No suele utilizarse.

Lista (Ready): Están listas para ejecutarse, a la espera de ser elegidas por el scheduler

Transiciones y funciones asociadas a ellas:

- Running → Blocked: por espera de evento (ej. `vTaskDelay()`, `xQueueReceive()`, `xSemaphoreTake()`).
- Blocked → Ready: el evento ocurre (tiempo expirado, dato disponible, semáforo dado) o se recibe una notificación.
- Running → Ready: el planificador la desaloja (fin de quantum o tarea de mayor prioridad)

lista).

- Ready → Running: el planificador la selecciona.
- Running → Suspended: llamada a `vTaskSuspend()`.
- Suspended → Ready: llamada a `vTaskResume()` o `xTaskResumeFromISR()`.

c) Las Tareas de procesamiento continuo (o cíclicas) son aquellas que ejecutan su trabajo repetidamente sin esperar eventos externos, o sea son tareas que no se bloquean. Suelen asignarse a prioridades bajas para no opacar indefinidamente a otras tareas.

Las tareas manejadas por eventos, en cambio, permanecen en estado bloqueado hasta que ocurre un evento (llegada de datos, pulsación de botón, etc.) y entonces ejecutan una acción. Pueden tener cualquier prioridad, pues no consumen CPU mientras esperan.

d) Las tareas pueden entrar en el estado Bloqueado para esperar dos tipos de eventos:

1. Eventos temporales (relacionados con el tiempo), donde los eventos son la finalización de un plazo o que se llega a un tiempo absoluto. Por ejemplo, una tarea puede entrar en estado Bloqueado a esperar que pasen 10 milisegundos.
2. Eventos de sincronización, donde los eventos se originan en otra tarea o interrupción. Por ejemplo, disponibilidad de datos en una cola, semáforos, grupos de eventos o notificaciones directas.

e) La tarea IDLE es creada automáticamente por FreeRTOS al iniciar el planificador. Tiene prioridad 0 (la más baja) y se ejecuta cuando no hay otras tareas listas. Sus funciones principales son:

- Mantener el procesador ocupado (evita que el CPU no ejecute nada, lo cual no es posible en un sistema con planificador).
- Liberar la memoria y recursos de las tareas eliminadas
- Medir el tiempo libre de CPU.
- Ejecutar rutinas de bajo consumo para ahorrar energía.

El procesador siempre está ejecutando alguna tarea; en ausencia de tareas de usuario, la IDLE task ocupa el CPU.

3) Una cola en FreeRTOS es un buffer FIFO (First In, First Out) que almacena un número fijo de elementos de tamaño fijo.

- La longitud y el tamaño de cada elemento se definen al crear la cola con `xQueueCreate()`.
- El kernel implementa la cola por copia: al enviar datos con `xQueueSend()`, se copia el contenido byte a byte en el buffer interno de la cola.
- Alternativamente, se puede usar una cola de punteros para transferir referencias a datos más grandes, evitando copias masivas y ahorrando RAM y tiempo de CPU.

4) En Es común tener varias tareas que producen datos y una sola que los procesa o registra, porque:

- Centraliza el procesamiento y evita condiciones de carrera.

- El lector puede tener prioridad mayor para consumir los datos rápidamente.
- Las colas manejan automáticamente la sincronización.

Ejemplo 2 tareas asociadas a distintos sensores envían lecturas a una tarea de carga a flash:

```

1  QueueHandle_t xColaDatos;
2
3  // Tarea escritora 1 (prioridad baja)
4  void vEscrivor1(void *pvParameters) {
5      int dato;
6      while(1) {
7          /* Lee sensor de analógico */
8          dato = /* valor leído */;
9          // Envía a la cola; si está llena se bloquea hasta que haya espacio
10         xQueueSend(xColaDatos, &dato, portMAX_DELAY);
11         vTaskDelay(pdMS_TO_TICKS(50));
12     }
13 }
14
15 // Tarea escritora 2 (prioridad baja)
16 void vEscrivor2(void *pvParameters) {
17     int dato;
18     while(1) {
19         /* Lee sensor de presión protocolo serie */
20         dato = /* valor leído */;
21         xQueueSend(xColaDatos, &dato, portMAX_DELAY);
22         vTaskDelay(pdMS_TO_TICKS(120));
23     }
24 }
25
26 // Única tarea lectora (prioridad más alta)
27 void vLector(void *pvParameters) {
28     int datoRecibido;
29     while(1) {
30         // Espera dato; si la cola está vacía se bloquea
31         if (xQueueReceive(xColaDatos, &datoRecibido, portMAX_DELAY) == pdTRUE) {
32             /* Guarda el dato en memoria SD */
33         }
34     }
35 }
36
37 // En main:
38 xColaDatos = xQueueCreate(10, sizeof(int));

```

Bloqueo por lectura: si la cola está vacía y una tarea la intenta leer, la tarea pasa a estado Blocked hasta que haya datos o expire el tiempo. Cuando llegan datos, se desbloquea la tarea de mayor prioridad que esté esperando; si hay varias con igual prioridad, la que lleva más tiempo esperando.

Bloqueo por escritura: si la cola está llena, una tarea que la intente escribir pasará a Blocked hasta que haya espacio o venza el tiempo. Se desbloquea a la tarea de mayor prioridad que espera espacio; si hay varias con igual prioridad, la que lleva más tiempo esperando.

- 5) Para Para enviar datos compuestos (conjuntos de datos de varios tipos) se crea la cola con el tamaño de una estructura que contenga todos esos tipos de datos y se envían los datos por copia. Esto no genera conflictos porque el tamaño se fija en tiempo de creación y este tamaño es coincidente con el tamaño de la estructura, luego el kernel gestiona la copia

internamente.

Si los datos son grandes (por ejemplo, una imagen o un buffer de varios KB), copiar cada elemento en la cola sería ineficiente y consumiría mucha RAM. La solución es usar una cola de punteros al dato, enviando solo la dirección de memoria donde se encuentra el dato. El emisor y el receptor deben acordar la gestión de la memoria (por ejemplo, usando un pool de buffers o malloc/free). La cola se crea con `xQueueCreate( longitud, sizeof(void*) )` y se envían punteros mediante `xQueueSend(..., &puntero, ...)`.

- 6) El esquema representa un sistema basado en tareas independientes que se comunican mediante una cola de mensajes. La cola almacena estructuras del tipo `Data_t`, compuestas por un identificador `eDataID`, que indica el tipo de dato recibido, y un valor `IDataValue`, que contiene la información.

Diferentes tareas del sistema actúan como productores de datos. Por ejemplo, la tarea del bus CAN recibe mensajes externos, los interpreta y genera una estructura `Data_t` con la información correspondiente, como puede ser la velocidad del motor. De forma similar, la tarea HMI genera mensajes cuando el usuario ingresa una acción o comando, identificándolos mediante otro valor de `eDataID`, como un cambio de referencia de velocidad.

Todas estas tareas envían sus mensajes a la cola sin necesidad de comunicarse directamente entre ellas. La tarea del controlador funciona como consumidor de la cola: toma los mensajes recibidos, analiza el identificador del dato y ejecuta la acción correspondiente según el tipo de información recibida.

Este método permite desacoplar las tareas del sistema, ya que cada una puede ejecutarse de forma independiente y el controlador recibe los datos de manera ordenada y segura mediante la cola.

- 7) Los Semáforos Mutex se utilizan para proteger el acceso a un recurso compartido por varias tareas. En el código a continuación se tienen dos tareas que accederían a un recurso compartido y con el uso de un semáforo mutex se garantiza que solo una tarea utilice dicho recurso a la vez y evitando que haya superposición de mensajes o lecturas de datos a medio modificar:

```

1  typedef struct {
2      uint16_t umbralAltaTemperatura;
3      uint16_t umbralBajaTemperatura;
4      uint16_t intervaloMuestreoMs;
5  } ConfigSistema_t;
6  // Variable global protegida por mutex
7  ConfigSistema_t xConfigSistema = {85, 5, 100, 0, 0};
8  SemaphoreHandle_t xMutexConfig;
9  // Tarea 1: LEE la configuración para controlar el sistema
10 void vControlTemperatura(void *pvParameters) {
11     uint16_t umbralAlta, umbralBaja, intervalo;
12     //inicializa valores a un valor predeterminado
13     while(1) {
14         // Leer configuración de forma segura (copia local atómica)
15         if (xSemaphoreTake(xMutexConfig, pdMS_TO_TICKS(50)) == pdTRUE) {
16             umbralAlta = xConfigSistema.umbralAltaTemperatura;
17             umbralBaja = xConfigSistema.umbralBajaTemperatura;
18             intervalo = xConfigSistema.intervaloMuestreoMs;
19             xSemaphoreGive(xMutexConfig);
20         } // Si no pudo tomar mutex, usa valores anteriores
21         /* Lee sensor de temperatura */
22         /* Compara con umbralAlta/umbralBaja */
23         /* Activa ventilador/calefactor según corresponda */
24         vTaskDelay(pdMS_TO_TICKS(intervalo));
25     }
26 }
27 // Tarea 2: MODIFICA la configuración por comandos (ej: UART)
28 void vActualizadorConfig(void *pvParameters)
29 {
30     uint16_t nuevoValor;
31     while(1) {
32         /* Espera recepción de comando*/
33         nuevoValor = /* valor extraído del mensaje */;
34         if (xSemaphoreTake(xMutexConfig, portMAX_DELAY) == pdTRUE) {
35             xConfigSistema.umbralAltaTemperatura = nuevoValor;
36             xSemaphoreGive(xMutexConfig);
37         }
38         vTaskDelay(pdMS_TO_TICKS(10));
39     }
40 }
41 // En main:
42 xMutexConfig = xSemaphoreCreateMutex();
43 xTaskCreate(vControlTemperatura, "Control", configMINIMAL_STACK_SIZE, NULL, 2, NULL);
44 xTaskCreate(vActualizadorConfig, "Actualiza", configMINIMAL_STACK_SIZE, NULL, 1, NULL);

```

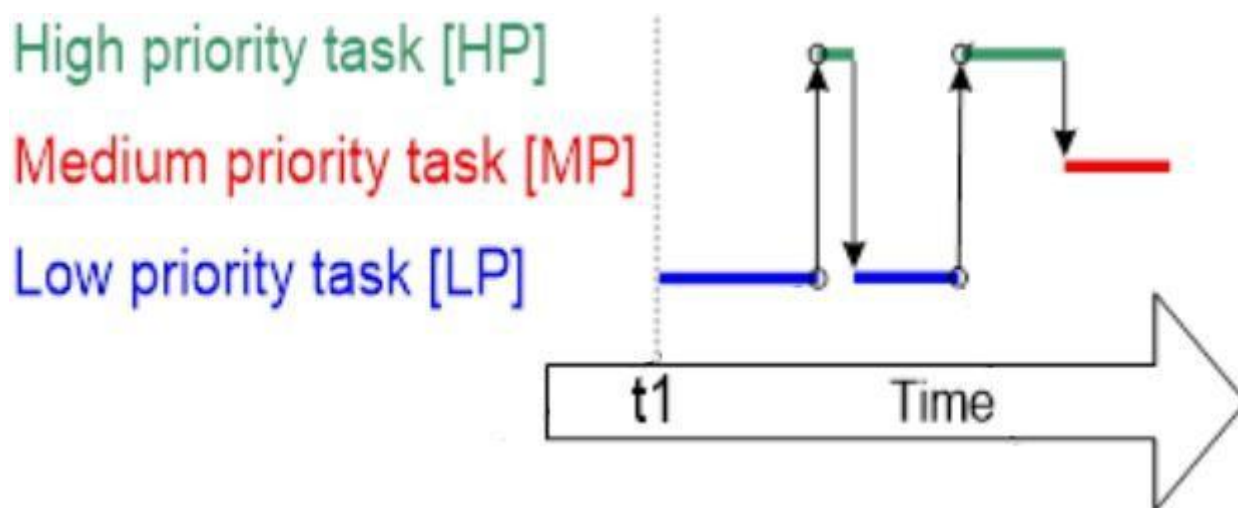
- 8) Este fenómeno ocurre cuando una tarea de baja prioridad mantiene un recurso compartido que necesita una tarea de alta prioridad, pero la tarea de alta prioridad no puede continuar hasta que el recurso esté libre. Si una tarea de prioridad media se ejecuta durante mucho tiempo, puede evitar que la tarea de baja prioridad libere el recurso, lo que genera un “bloqueo” indirecto de la tarea de alta prioridad. En el código a continuación cuando la tarea de alta prioridad queda bloqueada esperando un recurso que tiene la tarea de baja prioridad, FreeRTOS eleva temporalmente la prioridad de la tarea de baja prioridad al nivel de la tarea de alta prioridad, para que libere el recurso más rápidamente y así se evite el bloqueo, esto debido a que FreeRTOS usa herencia de prioridad automáticamente si se usa un mutex.

```

1 SemaphoreHandle_t xMutex;
2 // Estructura compartida protegida por mutex
3 struct {
4     int param1;
5     int param2;
6 } xConfiguracion = {0, 0}; // Inicialización
7 void TareaBajaPrioridad(void* pvParameters) {
8     while (1) {
9         printf("BAJA: tomando mutex...\n");
10        xSemaphoreTake(xMutex, portMAX_DELAY);
11        printf("BAJA: mutex tomado, trabajando...\n");
12        xConfiguracion.param1 = /* lee valor de RAM */;
13        xConfiguracion.param2 = /* lee otro valor de RAM */;
14        /* operación larga como escribir configuración en flash */
15        printf("BAJA: liberando mutex.\n");
16        xSemaphoreGive(xMutex);
17    }
18 }
19 void TareaMediaPrioridad(void* pvParameters) {
20     while (1) {
21         /* procesamiento continuo que acapara CPU */
22     }
23 }
24 void TareaAltaPrioridad(void* pvParameters) {
25     int p1, p2;
26     while (1) {
27         printf("ALTA: intentando tomar mutex...\n");
28         xSemaphoreTake(xMutex, portMAX_DELAY);
29         printf("ALTA: mutex tomado.\n");
30         /* operación breve: leer configuración */
31         p1 = xConfiguracion.param1;
32         p2 = xConfiguracion.param2;
33         printf("ALTA: liberando mutex.\n");
34         xSemaphoreGive(xMutex);
35         /* usa los parámetros leídos y calcula valores de ajuste para el sistema */
36     }
37 }
38 int main(void) {
39     xMutex = xSemaphoreCreateMutex();
40     xTaskCreate(TareaBajaPrioridad, "Baja", 1000, NULL, 1, NULL);
41     xTaskCreate(TareaMediaPrioridad, "Media", 1000, NULL, 2, NULL);
42     xTaskCreate(TareaAltaPrioridad, "Alta", 1000, NULL, 3, NULL);
43     while(1);
44 }

```

- 9) La tarea de baja prioridad (LP) toma mutex y después de un tiempo deja de ejecutarse por haber usado todos los slices de tiempo que le asignó el scheduler, la tarea de alta prioridad (HP) intenta tomar el mutex pero no puede porque LP aun no libera el mutex, por lo que HP se bloquea esperando a que el mutex esté disponible, si seguimos la política de scheduling al pie de la letra, la tarea de prioridad media (MP) debería ejecutarse luego de que HP se bloquea y hacer lo suyo hasta que habilite a que se ejecute LP. Con inversión de prioridades se hace una excepción a esto y la tarea LP hereda la prioridad de HP para prevenir que MP se ejecute e impida a LP devolver el mutex, una vez devuelto el mutex LP vuelve a su prioridad original. HP se ejecuta y ahora puede tomar el mutex, y una vez terminado lo que debía hacer con el recurso compartido devuelve el mutex y se bloquea dando lugar a que se ejecute MP.



- 10) Un punto muerto sucede cuando dos tareas quedan bloqueadas simultáneamente esperando el acceso a una cola compartida. En el código siguiente muestra 2 tareas las cuales quedan bloqueadas esperando un recurso (MutexA, MutexB) de la otra tarea.

```

1 SemaphoreHandle_t xMutexA, xMutexB;
2
3 /* Tarea 1: toma A, luego B */
4 void vTareaAlfa(void *pvParams) {
5     while(1) {
6         xSemaphoreTake(xMutexA, portMAX_DELAY);
7         /*
8          hace algo
9          */
10        xSemaphoreTake(xMutexB, portMAX_DELAY); // puede bloquearse si Beta tiene B
11        // hace algo usando ambos recursos
12        xSemaphoreGive(xMutexB);
13        xSemaphoreGive(xMutexA);
14        vTaskDelay(pdMS_TO_TICKS(1000));
15    }
16 }
17
18 /* Tarea 2: toma B, luego A (orden inverso) */
19 void vTareaBeta(void *pvParams) {
20     while(1) {
21         xSemaphoreTake(xMutexB, portMAX_DELAY);
22         /*
23          hace algo
24          */
25         xSemaphoreTake(xMutexA, portMAX_DELAY); // puede bloquearse si Alfa tiene A
26         // hace algo usando ambos recursos
27         xSemaphoreGive(xMutexA);
28         xSemaphoreGive(xMutexB);
29         vTaskDelay(pdMS_TO_TICKS(1000));
30     }
31 }

```

Suponiendo que:

- ambas tienen la misma prioridad
- luego de tomar MutexA y mientras tarea alfa hacia algo pasa a ejecutarse beta

El programa tomaría el siguiente curso: entraría a ejecutarse tarea beta, esta tarea tomaría el MutexB, ejecuta su código hasta llegar a semaphoretake de MutexA, donde se bloquearía porque tarea alfa tiene tomado MutexA, luego resumiría su ejecución tarea alfa hasta llegar al semaphoretake de MutexB donde se bloquearía por tener el mutex tomado tarea beta.

Finalmente, ambas se hallan bloqueadas a la espera de un evento que necesitan de la otra para desbloquearse.